NHIỀU CỬA SỔ & FRAME LỒNG NHAU

Selenium + pytest

Nhóm W4 - N07

Thành viên

Nguyễn Tiến Toàn - A51878
Nguyễn Trung Đức - A52093
Đào Quang Khải - A51922
Nguyễn Anh Tuấn - A51709
Phạm Bảo Phúc - A52197

MỤC TIÊU & TRANG THỰC HÀNH

MỤC TIÊU

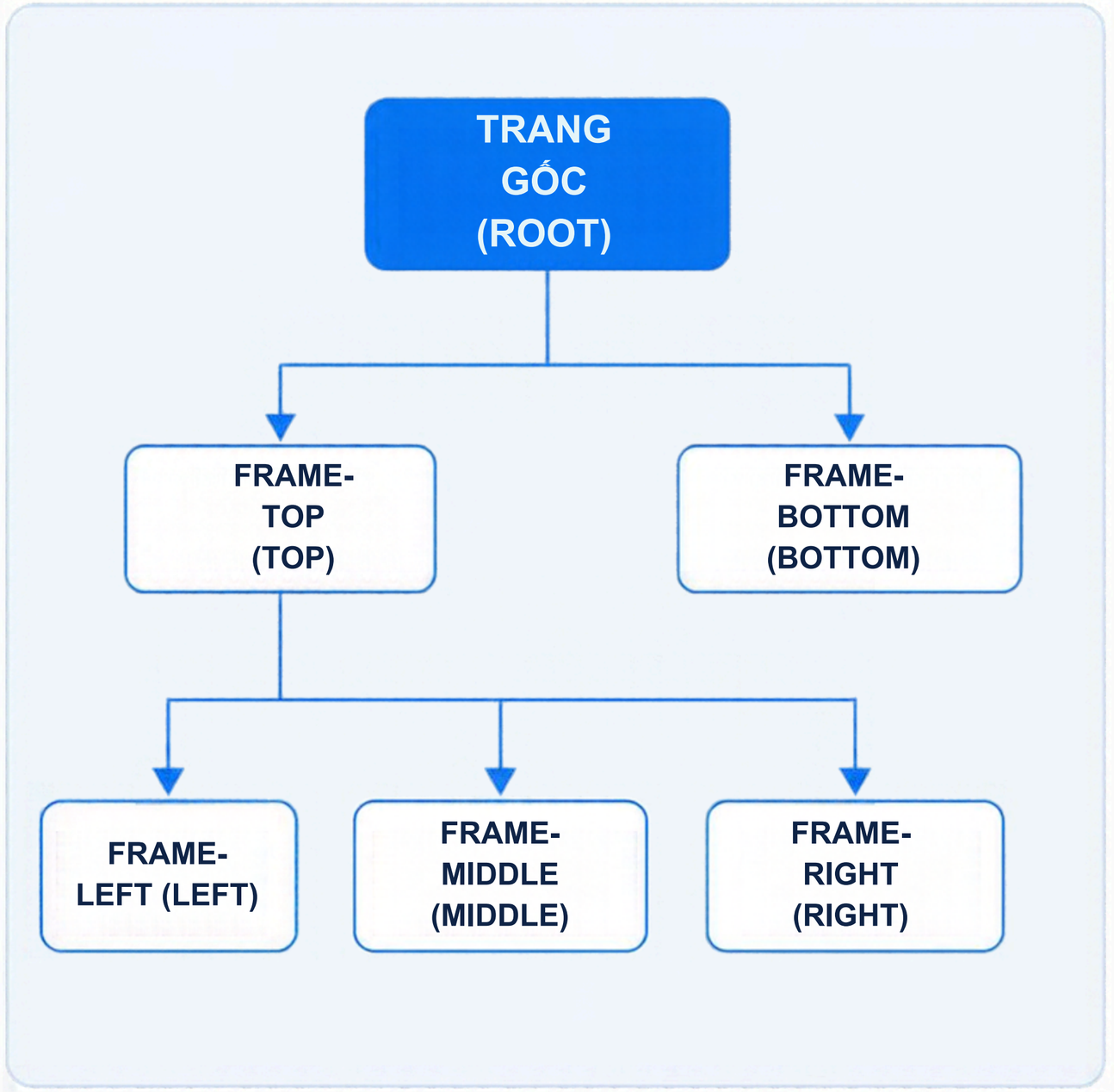
Điều khiển nhiều cửa sổ/tab và
frame lồng nhau băng Selenium

TRANG THỰC HÀNH

the-internet.herokuapp.com/windows
the-internet.herokuapp.com/nested_frame

TÀI LIỆU THAM KHẢO

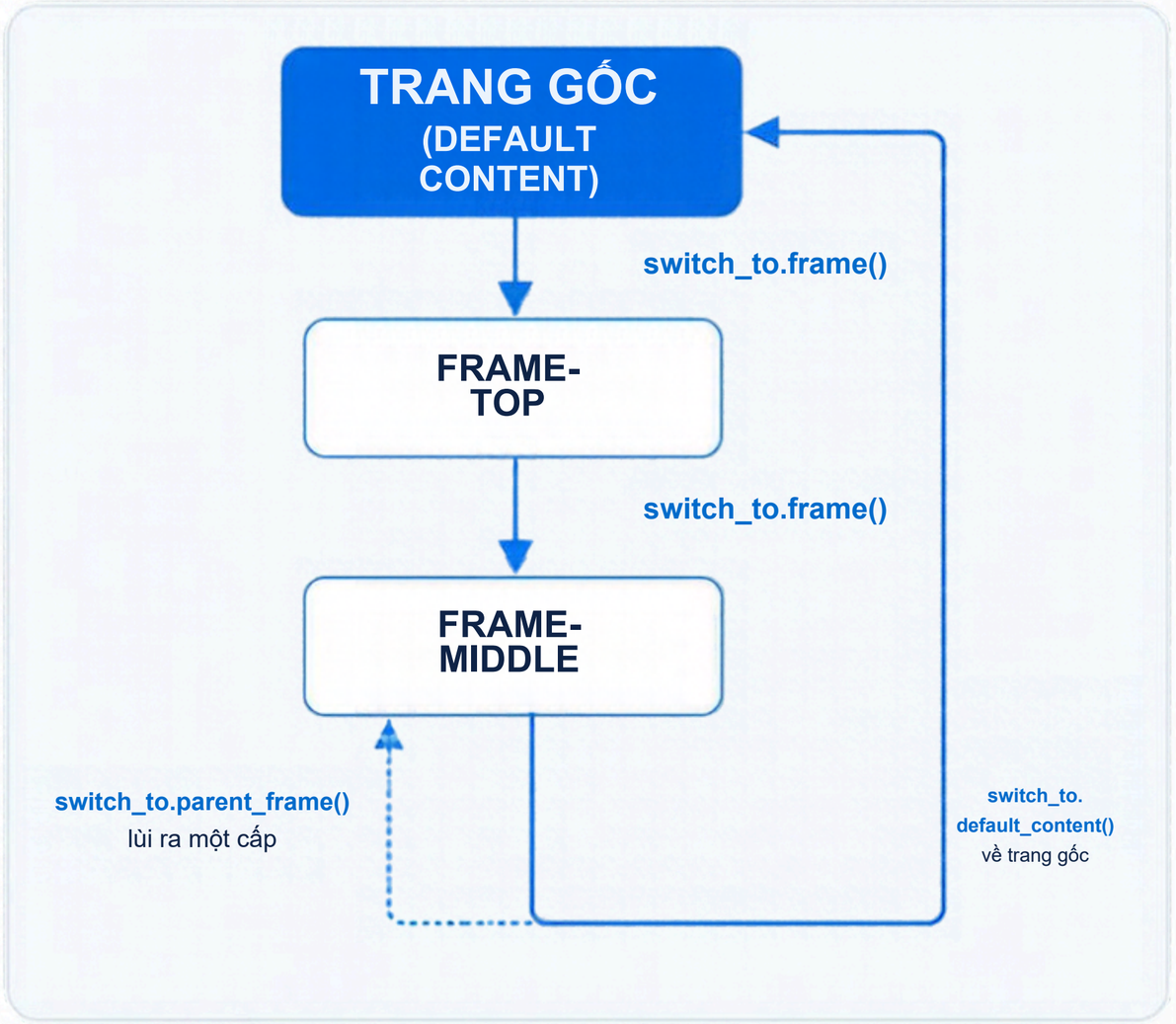
3.4, 3.6, 3.7, 7.19, 5.1, 7.1 (selenium-python)



NHIỀU CỬA SỔ

HÀM	Ý NGHĨA
WINDOW_HANDLES	LẤY DANH SÁCH ID CỦA TẤT CẢ CỬA SỔ/TAB ĐANG ĐƯỢC WEBDRIVER QUẢN LÝ
CURRENT_WINDOW_HANDLE	LẤY ID CỦA CỬA SỔ/TAB HIỆN TẠI
SWITCH_TO.WINDOW(HANDLE)	CHUYỂN SANG CỬA SỔ/TAB CÓ ID TƯƠNG ỨNG
EC.NUMBER_OF_WINDOWS_TO_BE()	CHỜ SỐ LƯỢNG CỬA SỔ/TAB BẰNG N

FRAME LỒNG NHAU – MỤC 3.4



HÀM	MÔ TẢ
switch_to.frame(x)	Chuyển vào frame x trước khi tìm phần tử bên trong
switch_to.parent_frame()	Lùi ra một cấp về frame cha (nếu có)
switch_to.default_content()	Về thẳng trang gốc (default content), thoát khỏi mọi frame

THÔNG TIN CƠ BẢN VỀ WAITS

1. Tổng quan về Waits

Mục đích: Đồng bộ tốc độ xử lý của kịch bản kiểm thử (test script) với tốc độ tải/phản hồi của trang web.

Tác dụng: Tránh lỗi không tìm thấy phần tử (NoSuchElementException) khi giao diện chưa kịp tải xong.

3 loại Waits chính: Implicit Wait, Explicit Wait và Fluent Wait.

BẢNG SO SÁNH 3 LOẠI WAIT TRONG SELENIUM	
Implicit Wait (Chờ ngầm định)	Chờ phần tử xuất hiện trong DOM (Presence)
Explicit Wait (Chờ tường minh)	Chờ theo trạng thái cụ thể (Visible, Clickable, Frame available,...)
Fluent Wait (Chờ linh hoạt nâng cao)	Chờ theo điều kiện linh hoạt + tùy chỉnh tần suất lặp/bỏ qua lỗi.

EXPLICIT WAITS & EXCEPTIONS – MỤC 5.1, 7.1

EXPLICIT WAITS

Là cơ chế chờ trong Selenium, cho phép chương trình đợi đến khi một điều kiện cụ thể được thỏa mãn hoặc hết thời gian chờ mới tiếp tục thực hiện.

EXCEPTIONS THƯỜNG GẶP	
NoSuchFrameException	sai tên frame hoặc quên default_content()
NoSuchElementException	tìm phần tử khi chưa vào frame
NoSuchWindowException	dùng driver sau close()
TimeoutException	hết giờ chờ

HISTORY, COOKIES, REMOTE WEBDRIVER - MỤC 3.6, 3.7, 7.19

HISTORY

> back() / forward(): Điều hướng trang

> current_url: Lấy URL đang mở

COOKIES

> add_cookie / get_cookie

> cookie dùng chung giữa các tab cùng domain

REMOTE WEBDRIVER

```
webdriver.Remote(
  command_executor='http://host:4444'
)
```

> chạy trên Selenium Grid

> Code test giữ nguyên

> hỗ trợ đầy đủ API windows, frames, cookies giống local driver.

CODE TEST CỦA NHÓM

```python
# BÀI 1: KIỂM THỬ XỬ LÝ WINDOWS (CỬA SỔ VÀ TAB MỚI)
# 3.4 Windows and frames - window_handles, current_window_handle, switch_to.window
# 5.1 Explicit Waits - EC.number_of_windows_to_be
def test_windows_handling():
    driver = webdriver.Chrome()
    driver.get("https://the-internet.herokuapp.com/windows")

    # Khởi tạo bộ đếm thời gian chờ tối đa 10 giây
    wait = WebDriverWait(driver, 10)

    # current_window_handle
    # Lưu lại ID của cửa sổ gốc trước khi mở tab mới
    original_window = driver.current_window_handle

    # Click vào chữ "Click Here" để mở một tab mới
    driver.find_element(By.LINK_TEXT, "Click Here").click()

    # EC.number_of_windows_to_be()
    # Chờ cho đến khi tổng số cửa sổ trình duyệt đang mở bằng 2
    wait.until(EC.number_of_windows_to_be(2))

    # 3.4 Windows and frames - window_handles
    # Lấy danh sách ID của tất cả các cửa sổ trình duyệt đang mở
    for window in driver.window_handles:
        if window != original_window:
            # switch_to.window()
            # Chuyển sang tab mới vừa được mở
            driver.switch_to.window(window)
            break

    # Tìm thẻ h3 trên tab mới và kiểm tra nội dung hiển thị xem có đúng là "New Window" không
    element = driver.find_element(By.TAG_NAME, "h3")
    assert element.text == "New Window"

    driver.quit()
```

Kiểm thử xử lý Windows (cửa sổ & tab mới)
(Windows and frames & Explicit Waits)

```python
# BÀI 2: KIỂM THỬ XỬ LÝ FRAMES (KHUNG TRANG WEB)
# 3.4 Windows and frames - switch_to.frame, switch_to.parent_frame, switch_to.default_content
def test_frames_handling():
    driver = webdriver.Chrome()
    driver.get("https://the-internet.herokuapp.com/nested_frames")

    # switch_to.frame()
    # Chuyển vào frame có tên "frame-top" để thao tác với nội dung bên trong frame này
    driver.switch_to.frame("frame-top")

    # Tiếp tục chuyển vào frame con có tên "frame-middle" nằm bên trong frame-top
    driver.switch_to.frame("frame-middle")
    # Tìm phần tử có id="content" và kiểm tra nội dung văn bản có đúng là "MIDDLE" không
    assert driver.find_element(By.ID, "content").text == "MIDDLE"

    # switch_to.parent_frame()
    # Quay lại frame cha, tức là từ frame-middle trở về frame-top
    driver.switch_to.parent_frame()

    # switch_to.default_content()
    # Thoát khỏi tất cả các frame và quay trở lại trang HTML gốc ban đầu
    driver.switch_to.default_content()

    driver.quit()
```

Kiểm thử xử lý Frames
(Windows and frames)

CODE TEST CỦA NHÓM

```python
# BÀI 3: KẾT HỢP EXPLICIT WAITS VÀ FRAMES
# 3.4 Windows and frames - EC.frame_to_be_available_and_switch_to_it
# 5.1 Explicit Waits - EC.frame_to_be_available_and_switch_to_it
def test_frame_wait_and_switch():
    driver = webdriver.Chrome()
    driver.get("https://the-internet.herokuapp.com/nested_frames")

    wait = WebDriverWait(driver, 10)

    # EC.frame_to_be_available_and_switch_to_it()
    # Chờ cho đến khi frame có tên "frame-bottom" xuất hiện và sẵn sàng thao tác. Khi tìm thấy, Selenium sẽ tự động chuyển vào frame đó
    wait.until(EC.frame_to_be_available_and_switch_to_it("frame-bottom"))

    # Kiểm tra mã nguồn trang hiện tại có chứa chữ "BOTTOM" hay không
    assert "BOTTOM" in driver.page_source
    driver.quit()
```

Kết hợp Explicit Waits & Frames
(Windows and frames & Explicit Waits)

CODE TEST CỦA NHÓM

```python
# BÀI 4: KIỂM THỬ LỊCH SỬ VÀ ĐIỀU HƯỚNG TRÌNH DUYỆT
# 3.6 History and location - back, forward, current_url
def test_history_and_location():
    driver = webdriver.Chrome()

    # get()
    # Truy cập trang Windows trước, sau đó truy cập trang chủ
    # Việc này tạo ra lịch sử điều hướng gồm 2 trang đã truy cập
    driver.get("https://the-internet.herokuapp.com/windows")
    driver.get("https://the-internet.herokuapp.com/")

    # back()
    # Nhấn nút Back trên trình duyệt để quay lại trang đã truy cập trước đó
    driver.back()
    # current_url
    # Kiểm tra URL hiện tại có chứa chữ "windows" hay không
    assert "windows" in driver.current_url

    # forward()
    # Nhấn nút Forward trên trình duyệt để chuyển tiếp đến trang tiếp theo
    driver.forward()
    # Kiểm tra URL hiện tại có đúng bằng địa chỉ trang chủ hay không
    assert driver.current_url == "https://the-internet.herokuapp.com/"

    driver.quit()
```

Kiểm thử lịch sử & điều hướng trình duyệt (History and location)

CODE TEST CỦA NHÓM

```python
# BÀI 5: KIỂM THỬ QUẢN LÝ COOKIES
# 3.7 Cookies - add_cookie, get_cookies
def test_cookies():
    driver = webdriver.Chrome()
    driver.get("https://the-internet.herokuapp.com/windows")

    # add_cookie()
    # Thêm một cookie mới vào trình duyệt tên là "Heisenberg" với giá trị là "99.1%"
    driver.add_cookie({'name': 'Heisenberg', 'value': '99.1%'})

    # get_cookies()
    # Lấy danh sách toàn bộ cookies hiện có của trang web
    cookies = driver.get_cookies()

    # Kiểm tra trong danh sách cookies có cookie vừa thêm hay không
    assert any(c['name'] == 'Heisenberg' and c['value'] == '99.1%' for c in cookies) # vừa duyệt dữ liệu vừa kiểm tra

    driver.quit()
```

Kiểm thử quản lý Cookies

CODE TEST CỦA NHÓM

```python
# BÀI 6: KIỂM THỬ XỬ LÝ NGOẠI LỆ (EXCEPTIONS)
# 7.1 Exceptions - TimeoutException, try-except
# 5.1 Explicit Waits - EC.presence_of_element_located
def test_exceptions():
    driver = webdriver.Chrome()
    driver.get("https://the-internet.herokuapp.com/windows")

    wait = WebDriverWait(driver, 2)

    # try-except
    # Sử dụng try-except để bắt và xử lý lỗi phát sinh trong quá trình thực thi chương trình
    try:
        # EC.presence_of_element_located()
        # Cố tình chờ một phần tử có id="id_khong_ton_tai" không tồn tại trên trang web
        # Nếu hết 2 giây mà vẫn không tìm thấy, Selenium sẽ phát sinh lỗi TimeoutException
        wait.until(EC.presence_of_element_located(By.ID, "id_khong_ton_tai"))
    except TimeoutException:
        # TimeoutException
        # Bắt đúng loại lỗi TimeoutException khi thời gian chờ đã hết
        print("Đã bắt được lỗi do hết thời gian chờ.")
        pass

    driver.quit()
```

Kiểm thử xử lý ngoại lệ
(Exceptions & Explicit Waits)

CODE TEST CỦA NHÓM

```python
# BÀI 7: KIỂM THỬ KẾT NỐI TỪ XA (REMOTE WEBDRIVER)
# 7.19 Remote WebDriver - webdriver.Remote
def test_remote_webdriver():
    # Phải có Selenium Server đang chạy ở port 4444 thì đoạn này mới hoạt động
    try:
        options = webdriver.ChromeOptions()

        # webdriver.Remote()
        # Thay vì khởi tạo trình duyệt trực tiếp bằng webdriver.Chrome() thì ở đây sử dụng webdriver.Remote() để kết nối đến máy chủ Selen
        driver = webdriver.Remote(
            command_executor='http://127.0.0.1:4444/wd/hub',
            options=options
        )

        # Truy cập trang Windows thông qua trình duyệt được điều khiển từ xa
        driver.get("https://the-internet.herokuapp.com/windows")
        # Kiểm tra tiêu đề trang web có chứa chữ "The Internet" hay không
        assert "The Internet" in driver.title
        driver.quit()

    except Exception:
        print("Bỏ qua bài test này vì không có Selenium Server port 4444")
```

Kiểm thử kết nối từ xa
(Remote WebDriver)

KẾT QUẢ

Kết quả kiểm thử

```
PS C:\Users\Admin\Documents\A51878\Kiểm_thử\my_selenium_project> pytest test_windows_nestedframes.py
============================= test session starts ==============================
platform win32 -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Admin\Documents\A51878\Kiểm_thử\my_selenium_project
collected 7 items

test_windows_nestedframes.py .......                                    [100%]

============================== 7 passed in 78.84s (0:01:18) ==============================
PS C:\Users\Admin\Documents\A51878\Kiểm_thử\my_selenium_project>
```

Cảm ơn cô và các bạn đã lắng nghe

THANK YOU FOR LISTENING